

2° Instancia Evaluativa

Interfaz Gráfica

Proyecto: Simulador de Ecosistema — versión Swing

Información General

- Examen práctico grupal (3-4 integrantes)
- Entrega: 18 junio 2026

¿De qué se trata?

En la primera instancia evaluativa desarrollaron el Simulador de Ecosistema como una aplicación de consola en Java. En esta segunda instancia, el objetivo es transformar ese simulador en una aplicación de escritorio con interfaz gráfica, aplicando la arquitectura MVC + DAO y persistiendo los datos en archivos de texto.

En este proyecto puede realizar un panel gráfico ubicando en tiempo real a cada entidad (puede ser una grilla con símbolos o colores que diferencien lobo, conejo y planta).

Lo que se reutiliza de IE1:

- Entidad, Animal, Planta, Conejo, Lobo → pasan al paquete modelo sin cambios estructurales.
- Interfaces Reproducible y Mortal → se mantienen exactamente igual.
- Clase Ecosistema con toda su lógica de procesarTurno(), ecosistemaColapsado(), generarReporteFinal(), etc.
- La mecánica del juego: configuración inicial, loop por turnos, intervención cada 3 turnos, condiciones de fin.

Lo que se agrega:

- Arquitectura MVC: separación en Modelo, Vista, Controlador y DAO.
- Vista en Swing: interfaz gráfica diseñada en NetBeans que reemplaza la terminal.
- DAO: clases que leen y escriben el estado del ecosistema en archivos .txt.
- Persistencia: el estado se guarda al pausar o cerrar la simulación y se puede retomar.

Entregables obligatorios:

CRÍTICO: La falta de cualquiera de los 3 entregables obligatorios anula la instancia evaluativa del alumno

Código del proyecto (obligatorio)

- Link al repositorio de GitHub con el proyecto completo.
- TODOS los integrantes deben tener commits realizados.
 - **Penalización: integrante sin commits pierde el 50% de la nota grupal.**

Video explicativo (obligatorio)

- Duración: entre 10 y 15 minutos.
- Todos los integrantes con cámara encendida.
- Cada integrante debe mostrar su pantalla con el código al momento de explicar.
- Se debe explicar: la arquitectura MVC+DAO del proyecto, el código técnico de la parte implementada, los conflictos encontrados y cómo los resolvieron, y el uso transparente de IA u otras fuentes.
- Se evaluará especialmente la comprensión individual de cada parte del código implementada.
- Pueden usar Vimeo para capturar pantalla y rostro. Sin webcam: grabarse con el celular y unir con Canva.

Documentación (obligatorio)

- En la carpeta del proyecto debe haber una carpeta con capturas de los prompts utilizados, o un Word con los links a las conversaciones completas con la IA.
- Esta entrega es INDIVIDUAL. Deberán dejar claro de quién es cada captura.
- Captura del diseño de Figma.
- Alternativa: agregar los links de las conversaciones completas y/o diseño en un README.md

README.md (opcional, recomendado)

- Instrucciones de ejecución (versión de NetBeans, JDK, dependencias).
- Descripción de la arquitectura del proyecto y los paquetes.
- Integrantes y rol de cada uno.
- Desafíos encontrados y cómo se resolvieron.

Nota: esta es la parte práctica de la evaluación. El día de la entrega, cada integrante tendrá que realizar un cuestionario individual y entregar el link del repositorio.

Arquitectura obligatoria — MVC + DAO

El proyecto debe estar organizado en las cuatro capas vistas en clase con paquetes separados. No se aceptan proyectos donde la lógica de la simulación, la interfaz y el acceso a datos estén mezclados en los mismos archivos.

Paquete / Capa	Responsabilidad
modelo	Las clases de IE1: Entidad, Animal, Planta, Conejo, Lobo, Ecosistema, Reproducible, Mortal. Pueden adaptarse para la persistencia si es necesario.
dao	Clases que leen y escriben el estado del ecosistema (listas de entidades, turno actual, clima) en archivos .txt. No conocen la Vista ni el Controlador.
controlador	Recibe las acciones de la Vista (avanzar turno, cambiar clima, agregar entidad), ejecuta la lógica del Ecosistema, llama al DAO y actualiza la Vista. Sin código Swing.
vista	JFrame diseñado en NetBeans. Los listeners solo llaman al Controlador. No ejecuta lógica de la simulación directamente.

CRÍTICO: Violaciones de arquitectura que anulan el criterio:

- **FileWriter o FileReader dentro de un ActionListener o en la Vista.**
- **Lógica de la simulación (procesarTurno, comer, reproducirse, etc.) dentro de la Vista.**
- **El DAO con referencias a componentes Swing.**
- **El Controlador con imports de javax.swing o llamadas a JOptionPane.**

Persistencia obligatoria — Archivo de flujo

La simulación debe poder guardarse y retomarse. Al abrir la app se puede continuar una simulación anterior o iniciar una nueva.

- El estado del ecosistema (listas de entidades, turno actual, clima y contadores) se guarda en archivos .txt.
- Los datos se cargan al iniciar si existe una simulación guardada.
- Los datos se guardan al pausar o cerrar la simulación.
- Si no existe archivo guardado, la app arranca con la pantalla de configuración inicial.
- Los archivos se generan en la carpeta del proyecto, sin rutas absolutas.

Dos formas válidas de persistencia — elegir una:

- **Texto plano** (FileWriter + BufferedReader): agregar toTexto() y fromTexto() a las clases del Modelo.
- **Serialización** (ObjectOutputStream + ObjectInputStream): las clases implementan Serializable y se guarda el estado completo.

Diseño de interfaz — Figma

El diseño de la interfaz es parte del proyecto. Cada grupo debe diseñar en Figma cómo va a verse la aplicación antes de programarla, aplicando los conceptos de UX/UI vistos en clase.

La estética es libre — el grupo elige colores, tipografía y estilo visual. Lo que importa es que las decisiones de diseño estén justificadas por los principios aprendidos.

Entrega:

- El link al archivo de Figma se incluye en el repositorio o en el README junto con el resto del proyecto.
- El diseño no se entrega por separado — forma parte del entregable de código.

Componentes Swing obligatorios

Los siguientes componentes deben estar presentes y funcionalmente integrados — no solo colocados sin uso:

Componente	Uso esperado en la simulación
JLabel	Mostrar contadores en tiempo real, ej: cantidad de plantas, conejos y lobos vivos. Clima actual. Turno actual.
JTable	Listado de entidades vivas con columnas, ej: tipo, nombre, energía y edad. Se actualiza turno a turno.
JList	Log de eventos del turno, ej: quién comió a quién, quién nació, quién murió. Se limpia o acumula por turno.
JButton	Acciones principales: Avanzar turno, Pausar/Reanudar, Guardar, Nueva simulación. Al menos tres botones funcionales.
JMenuBar	Menú con al menos: Simulación (nueva, guardar, cargar) y Reportes (reporte final, estado actual).
JOptionPane	Confirmación antes de iniciar una nueva simulación si hay una en curso. Mensaje al colapsar el ecosistema.

CardLayout	Navegación entre al menos dos vistas: configuración inicial y vista de simulación en curso.
JScrollPane	Envolviendo la JTable y la JList de eventos para que tengan scroll.

Layouts

- BorderLayout como estructura principal de la ventana.
- Al menos un layout adicional aplicado con criterio dentro de algún panel (GridLayout, FlowLayout, BoxLayout o GridBagLayout).

Funcionalidades mínimas

La aplicación debe mostrar en pantalla la simulación en tiempo real. No es necesario replicar todas las funcionalidades de consola, pero sí las siguientes:

- Pantalla de configuración — formulario para ingresar cantidad inicial de entidades, clima y turnos totales, con las mismas validaciones de IE1.
- Avanzar turno — botón que ejecuta procesarTurno() y actualiza todos los paneles visibles.
- Panel de estado — JLabel con contadores actualizados: plantas, conejos, lobos vivos, clima y turno actual.
- Tabla de entidades — muestra las entidades vivas con sus datos. Se actualiza cada turno.
- Log de eventos — JList con los eventos del turno: muertes, nacimientos, cacerías.
- Intervención cada 3 turnos — JOptionPane o panel especial que permita cambiar clima o agregar entidades.
- Guardar y cargar — persistir el estado de la simulación y poder retomarlo.
- Reporte final — al terminar la simulación (por turnos o colapso), mostrar el reporte completo accesible desde el menú.
- Estado vacío — mensaje cuando no hay simulación en curso.

BONUS (puntos extras)

BONUS: Los puntos obtenidos en los ítems extras(al igual que la nota conceptual) se suman al puntaje final del proyecto para mejorar la nota global de la IE

JDBC — Agregar un DAO de base de datos

- Crear una base de datos MySQL con las tablas correspondientes al estado del ecosistema.
- Crear un DAO adicional exclusivo para MySQL que use Connection, PreparedStatement y ResultSet.
- El Modelo, el Controlador y la Vista no deben modificarse.
- Incluir en el README las instrucciones para crear la base de datos (script SQL o capturas).

Visualización de estadísticas en tiempo real

- Gráfico o tabla que muestre la evolución de las poblaciones turno a turno (historial de conteos).
- Indicar en qué turno cada población llegó a su máximo y mínimo.

Prototipo funcional en Figma

- Convertir el diseño en un prototipo navegable: conectar las pantallas con interacciones reales en Figma.
- El prototipo debe permitir simular la navegación entre configuración, simulación en curso y reporte final.

PlantaVenenosa en la interfaz

- Integrar la PlantaVenenosa de IE1 en la tabla de entidades con indicación visual diferenciada.
- El log de eventos debe registrar cuando un conejo come una planta venenosa.

README.md completo

- Instrucciones claras de ejecución, estructura del proyecto, integrantes con roles y desafíos.

Distribución de puntaje de la 2° IE

El puntaje se compone de dos partes que deben aprobarse de forma independiente:

- Cuestionario individual: 100 puntos → 40% de la nota global (mínimo para aprobar: 55/100)
- Proyecto grupal: 100 puntos → 60% de la nota global (mínimo para aprobar: 55/100)

Si se desaprueba cualquiera de las dos partes, la IE queda desaprobada con un 3 o menos.

Escala de valoración

Nota	Puntaje
1	01 – 24
2	25 – 39
3	40 – 54
4 (aprueba)	55 – 61
5	62 – 66
6	67 – 72
7 (accede a IEFI)	73 – 79
8	80 – 87
9	88 – 96
10	97 – 100